

Lecture (10)

Chapter 16: Exceptions

16.1 Exceptions

- An **exception** is a value or an object that indicates that an error has occurred
- When an exception occurs, the program must either terminate or jump to special code for handling the exception.
- The special code for handling the exception is called an **exception handler**

Exceptions – Key Words

- **throw** – followed by an argument, is used to signal an exception
- **try** – followed by a block { }, is used to invoke code that throws an exception
- **catch** – followed by a block { }, is used to process exceptions thrown in a preceding **try** block. It takes a parameter that matches the type of exception thrown

Throwing an Exception

- Code that detects the exception must pass information to the exception handler. This is done using a **throw** statement:

```
throw "Emergency!"  
throw 12;
```

- In C++, information thrown by the **throw** statement may be a value of any type

Catching an Exception

- Block of code that handles the exception is said to **catch** the exception and is called an **exception handler**
- An exception handler is written to catch exceptions of a given type: For example, the code

```
catch(char *str)
{
    cout << str;
}
```

can only catch exceptions of type C-string

Catching an Exception

Another example of a handler:

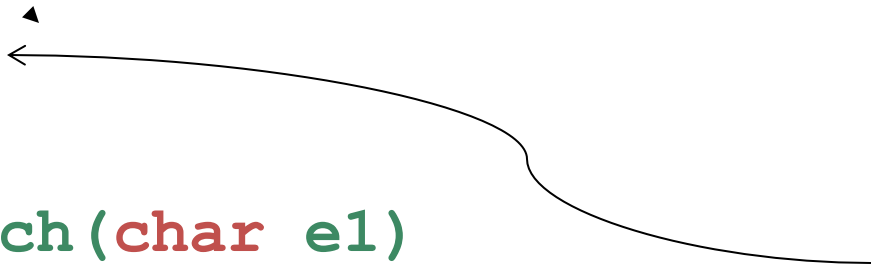
```
catch(int x)
{
    cerr << "Error: " << x;
}
```

This can catch exceptions of type `int`

Connecting to the Handler

Every catch block is attached to a **try** block of code and is responsible for handling exceptions thrown from that block

```
try
{
    ▲
    ←
}
catch(char e1)
{
    // This code handles exceptions
    // of type char that are thrown
    // in this block
}
```



Execution of Catch Blocks

- The catch block syntax is similar to that of a function
- A catch block has a formal parameter that is initialized to the value of the thrown exception before the block is executed

Exception Example

- An example of exception handling is code that computes the square root of a number.
- It throws an exception in the form of a C-string if the user enters a negative number

Example

```
int main( )
{
    try
    {
        double x;
        cout << "Enter a number: ";
        cin >> x;
        if (x < 0) throw "Bad argument!";
        cout << "Square root of " << x << " is " << sqrt(x) ;
    }
    catch(char *str)
    {
        cout << str;
    }
    return 0;
}
```

Flow of Control

1. Computer encounters a **throw** statement in a **try** block
2. The computer evaluates the **throw** expression, and immediately exits the **try** block
3. The computer selects an attached **catch** block that matches the type of the thrown value, places the value in the catch block's formal parameter, and executes the catch block

Uncaught Exception

- An exception may be uncaught if
 - there is no **catch** block with a data type that matches the exception that was thrown, or
 - it was not thrown from within a **try** block
- The program will terminate in either case

Handling Multiple Exceptions

Multiple catch blocks can be attached to the same block of code. The catch blocks should handle exceptions of different types

```
try{...}  
catch(int iEx){ }  
catch(char *strEx){ }  
catch(double dEx){ }
```

Throwing an Exception Class

- An **exception class** can be defined and thrown
- A catch block must be designed to catch an object of the exception class
- The exception class object can pass data to exception handler via data members

Exception When Calling **new**

- If **new** cannot allocate memory, it throws an exception of type **bad_alloc**
- Must **#include <new>** to use **bad_alloc**
- Can invoke **new** from within a **try** block, and use a **catch** block to detect that memory was not allocated.

Nested Exception Handling

try blocks can be nested in other **try** blocks and even in catch blocks

```
try
{
    try{ } catch(int i){ }
}
catch(char *s)
{ }
```

Where to Find an Exception Handler?

- The compiler looks for a suitable handler attached to an enclosing **try** block in the same function
- If there is no matching handler in the function, it terminates execution of the function, and continues the search for a handler starting at the point of the call in the calling function.

Unwinding the Stack

- An unhandled exception propagates backwards into the calling function and appears to be thrown at the point of the call
- The computer will keep terminating function calls and tracing backwards along the call chain until it finds an enclosing **try** block with a matching handler, or until the exception propagates out of **main** (terminating the program).
- This process is called **unwinding the call stack**

Rethrowing an Exception

- Sometimes an exception handler may need to do some tasks, then pass the exception to a handler in the calling environment.
- The statement

`throw;`

with no parameters can be used within a **catch** block to pass the exception to a handler in the outer block

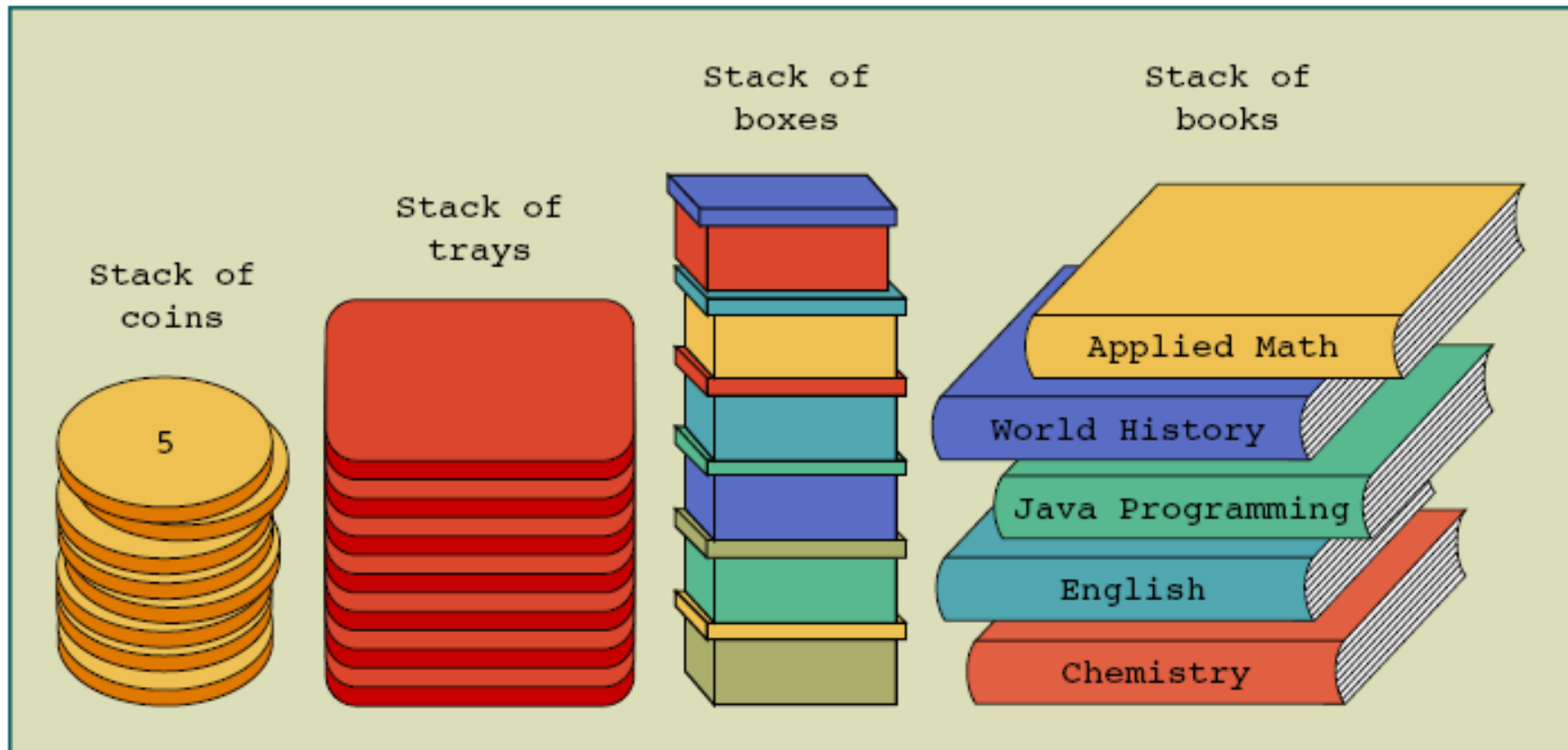
Lecture (10)

Chapter 18: Stacks and Queues

Stacks

- A **stack** is similar in concept to a pile of plates, books, blocks, boxes, etc.
 - The first item put on the stack is on the bottom of the stack.
 - All items added to the stack are placed on top of the previous item.
 - The last item put on the stack is on the top of the stack.
- Computers use stacks to implement method calls
- Real life examples : **The stack of trays in a cafeteria**; A stack of plates in a cupboard; A driveway that is only one car wide.

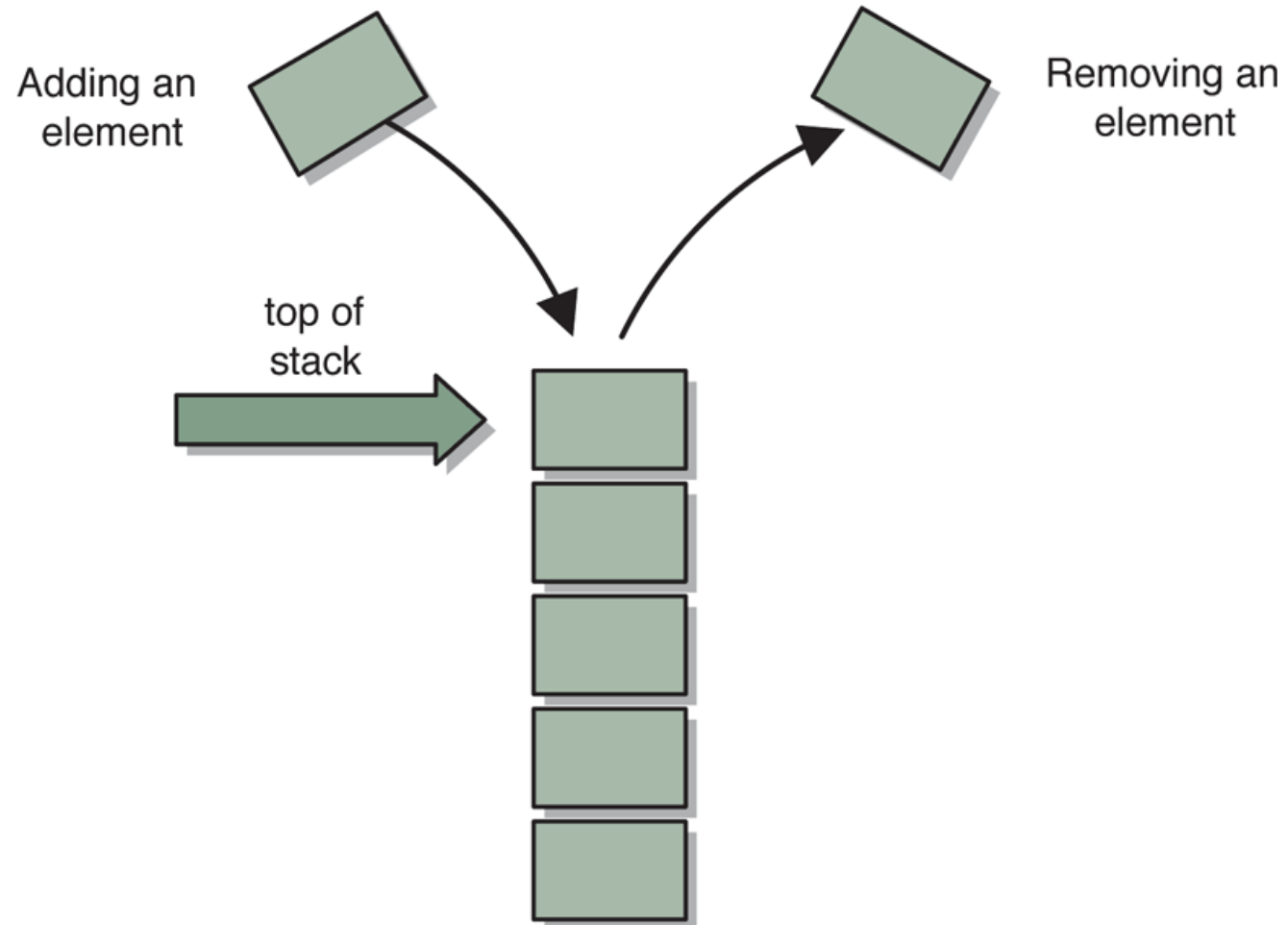
Stacks



Stacks

- Characteristics of **stacks**:
 - Data can only be placed on the top of the stack.
 - Data can only be removed from the top of the stack.
 - Data can only be removed from the bottom of the stack if there is only one item on the stack.
 - Data can **not** be removed from the middle of the stack without first removing all items from the top.

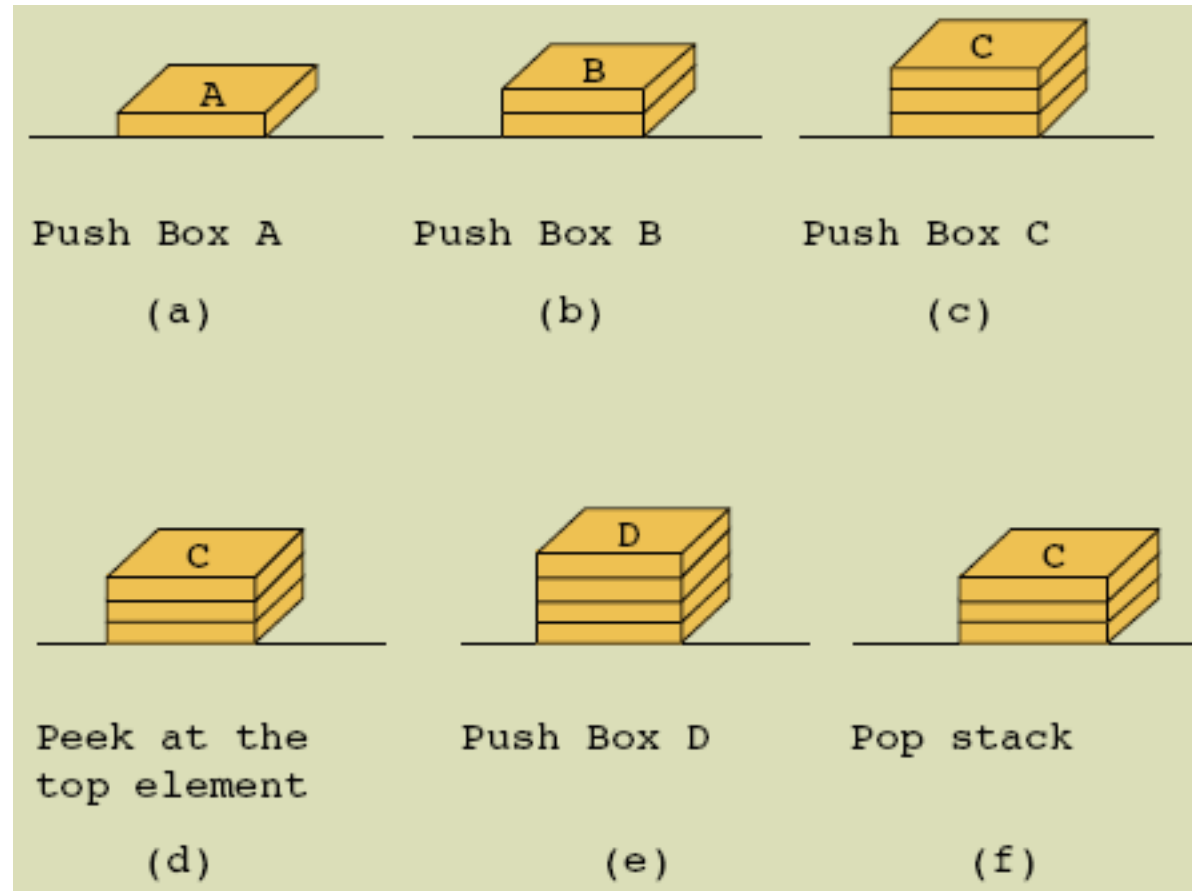
A conceptual view of a **stack**



Stacks

- **Stacks** are also called **Last In First Out (LIFO)** data structures
- Operations performed on stacks
 - **Push**: adds an element to the stack
 - **Pop**: removes an element from the stack
 - **Peek**: looks at the top element of the stack

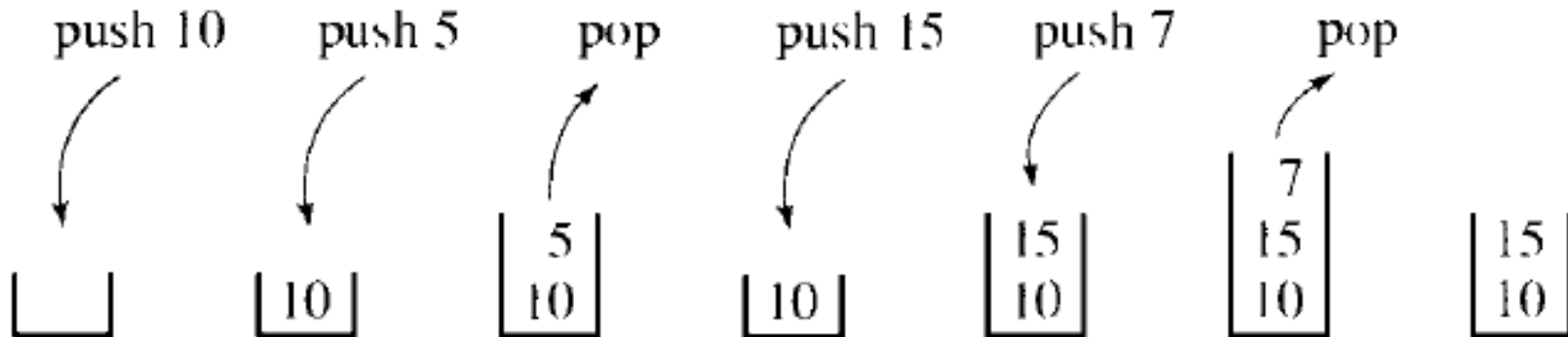
Stack operations



Stack Example

1. Try this sequence of operations

- push (10)
- push (5)
- pop
- push (15)
- push (7)
- pop



Stack – Question ???

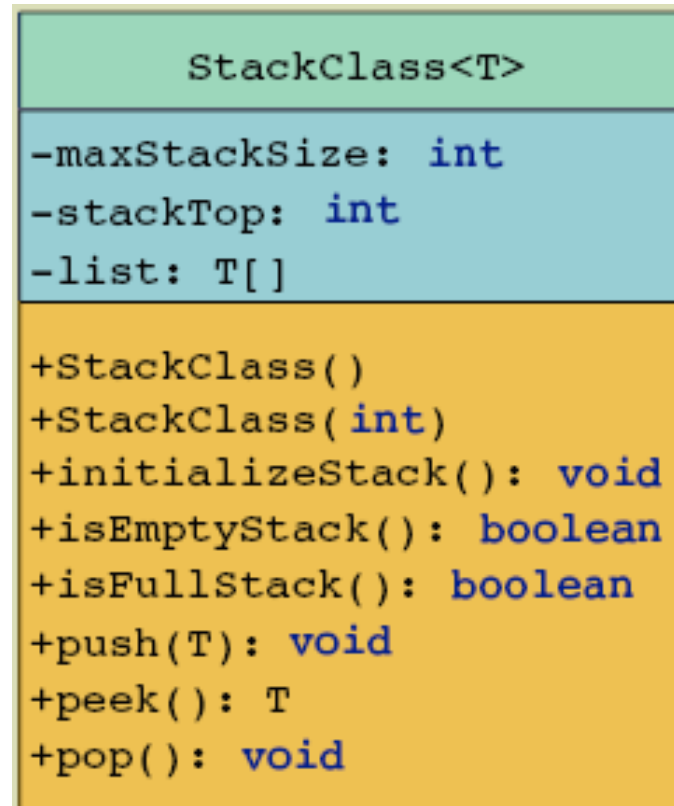
- What is the content of a stack after executing the following commands?
 - push(3)
 - push(6)
 - push(8)
 - push(1)
 - pop()
 - pop()
 - push(14)

				1	8	
			1			
		8	8	8		14
	6	6	6	6	6	6
3	3	3	3	3	3	3

Implementation of **Stacks** as **Arrays**

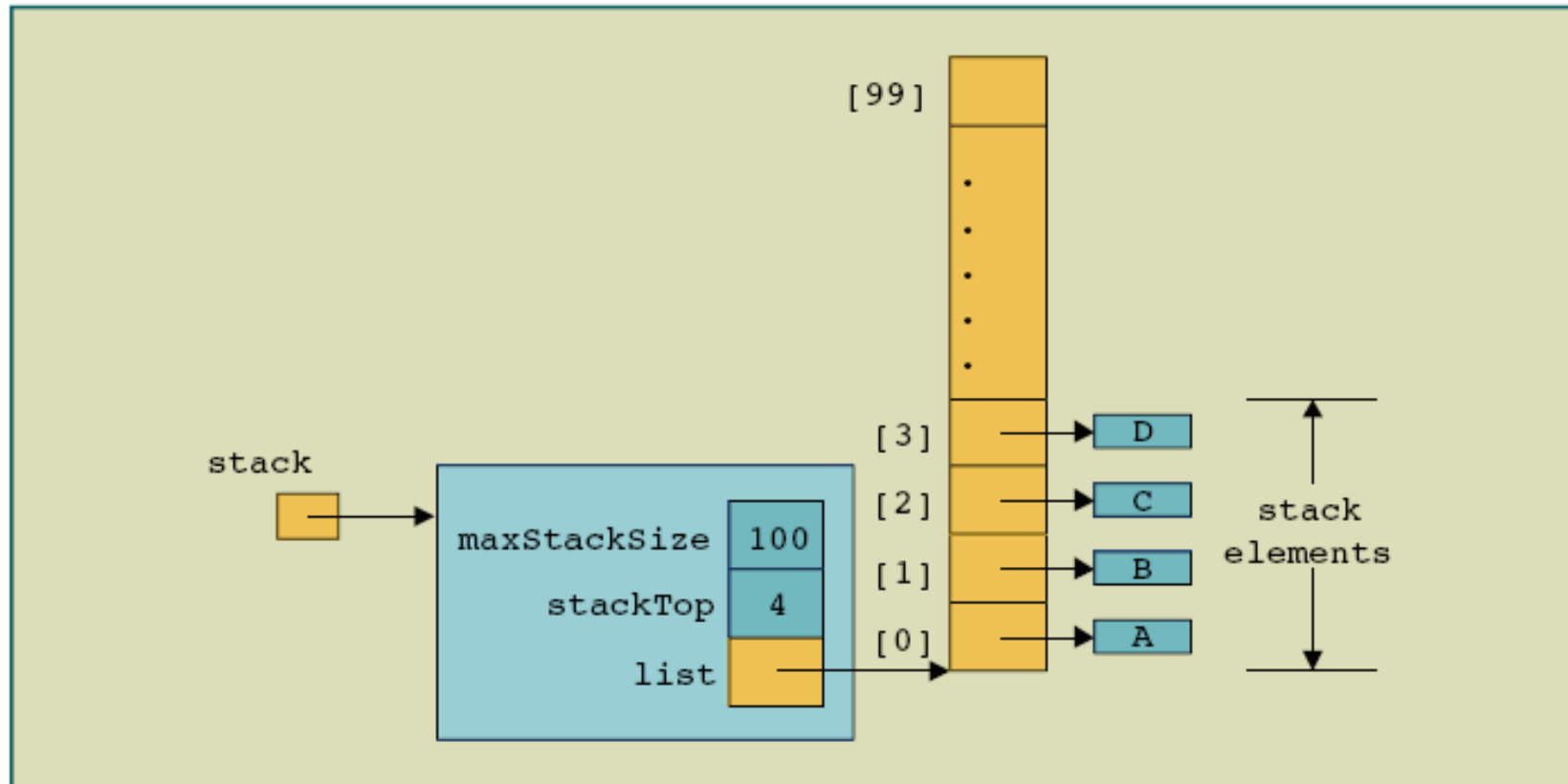
- The array implementing a stack is an array of reference variables
- Each element of the stack can be assigned to an array slot
- The top of the stack is the index of the last element added to the stack
- To keep track of the top position, declare a variable called **stackTop**

Implementation of **Stacks** as **Arrays**



UML class diagram of the **class** **StackClass**

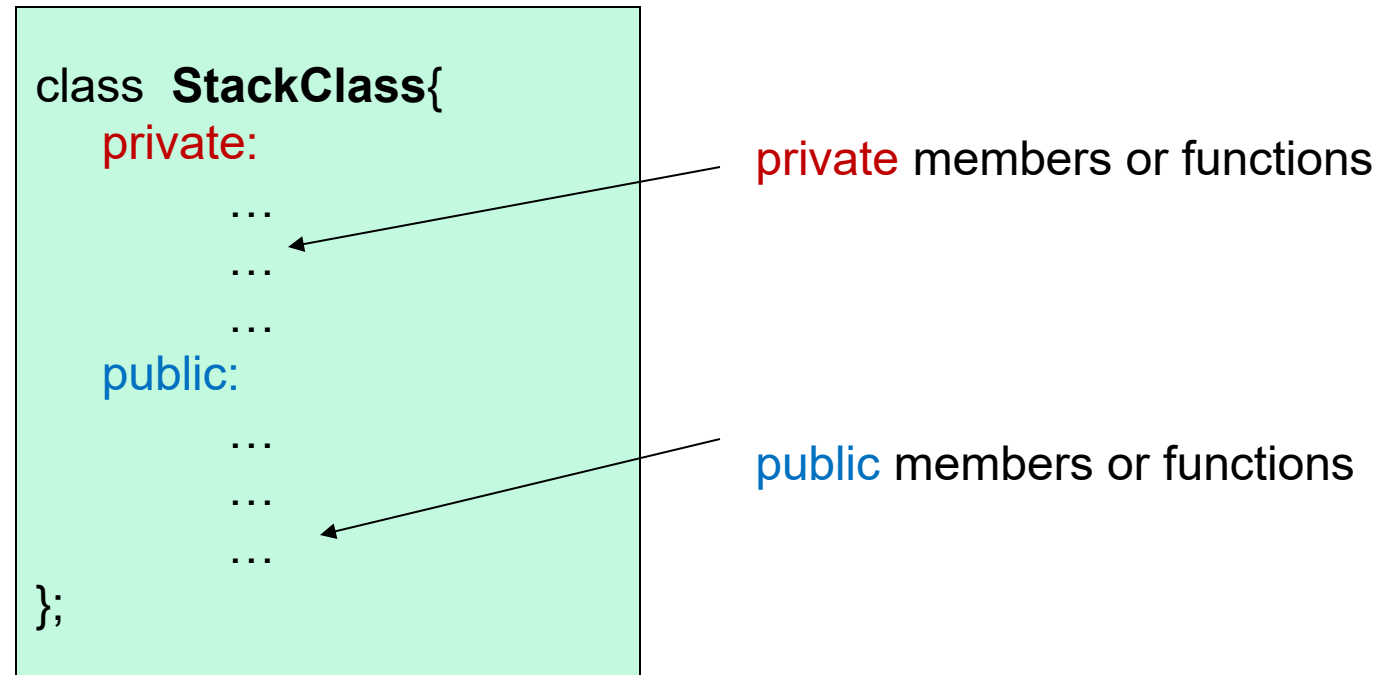
Implementation of **Stacks** as **Arrays**



Example of a stack

StackClass

-



StackClass – data members

```
class StackClass{  
    private:  
        int maxStackSize;  
        int stackTop;  
        int *List; //use any data type  
    public:  
        ...  
        ...  
        ...  
};
```

private data members

public member functions

(pre or user defined)

- You can use any

StackClass – member functions

- Default constructor

```
StackClass()  
{  
    maxSize = 100;  
    stackTop = -1; //set stackTop to -1  
    //create the array  
    list = new int[maxStackSize];  
}
```

- Constructor with parameter

```
StackClass(int S)  
{  
    maxSize = S;  
    stackTop = -1; //set stackTop to -1  
    //create the array  
    list = new int[maxStackSize];  
}
```

StackClass – member functions

- function **initializeStack**

```
void initializeStack()
{
    for (int i = 0; i < stackTop; i++)
        list[i] = NULL;
    stackTop = -1;
}
```

StackClass – member functions

- function **isEmptyStack**

```
bool isEmptyStack()  
{  
    return (stackTop == -1);  
}
```

- function **isFullStack**

```
bool isFullStack()  
{  
    return (stackTop == (maxStackSize-1));  
}
```

StackClass – member functions

- function **push**

```
void push(int newItem)
{
    if (isFullStack())
    {
        cout << "Cannot add->the stack is full" << endl;
        return;
    }
    stackTop++;           //increment stackTop
    list[stackTop] = newItem; //add newItem
}
```

StackClass – member functions

- function **pop**

```
int pop()
{
    if (isEmptyStack())
    {
        cout << "The stack is empty !!!" << endl;
        return -1;
    }
    return List[stackTop--];
}
```

StackClass – member functions

- function **peek**

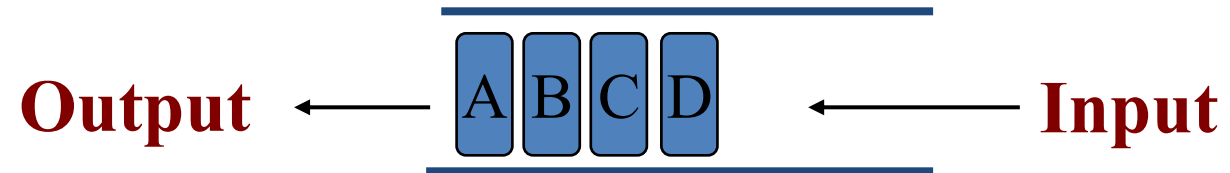
```
int peek()
{
    if (isEmptyStack())
    {
        cout << "The stack is empty !!!" << endl;
        return NULL;
    }
    return List[stackTop];
}
```

Queues

- ♦ Data structure in which the elements are added at one end, called the **rear**, and deleted from the other end, called the **front**.
- ♦ As in a stack, the middle elements of the queue are inaccessible.
- ♦ A queue is a **First In First Out (FIFO)** data structure.

Queues

- It is **FIFO** structure.



Queue Operations

- ◆ Queue operations

- ◆ initializeQueue

- ◆ isEmptyQueue

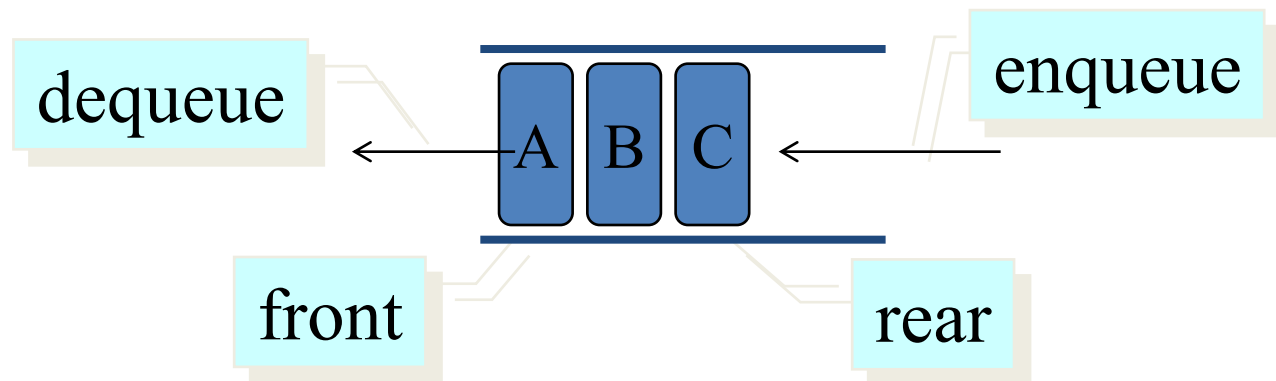
- ◆ isFullQueue

- ◆ front

- ◆ back

- ◆ addQueue (OR: enqueue)

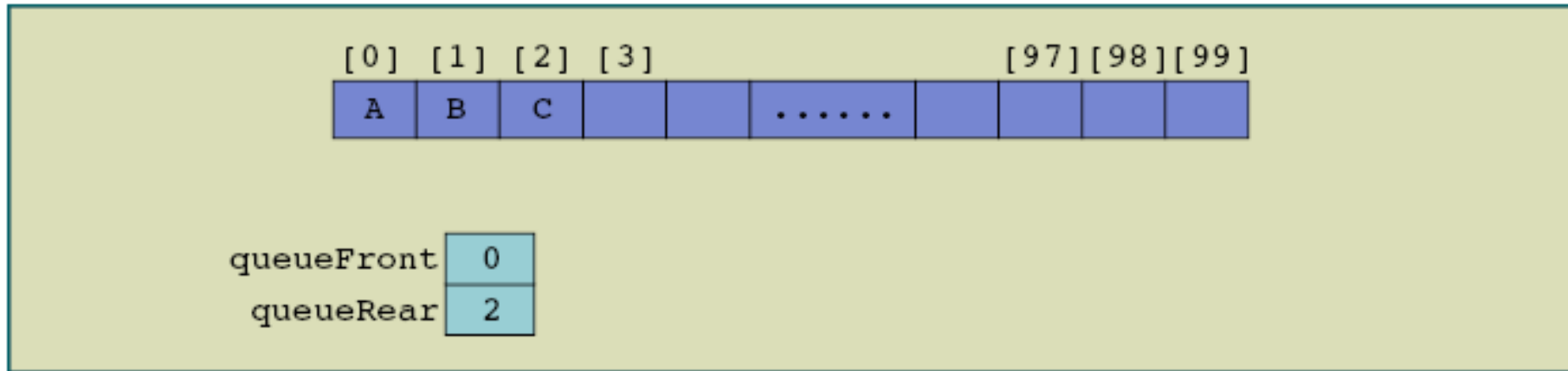
- ◆ deleteQueue (OR: dequeue)



Implementation of **Queues** as **Arrays**

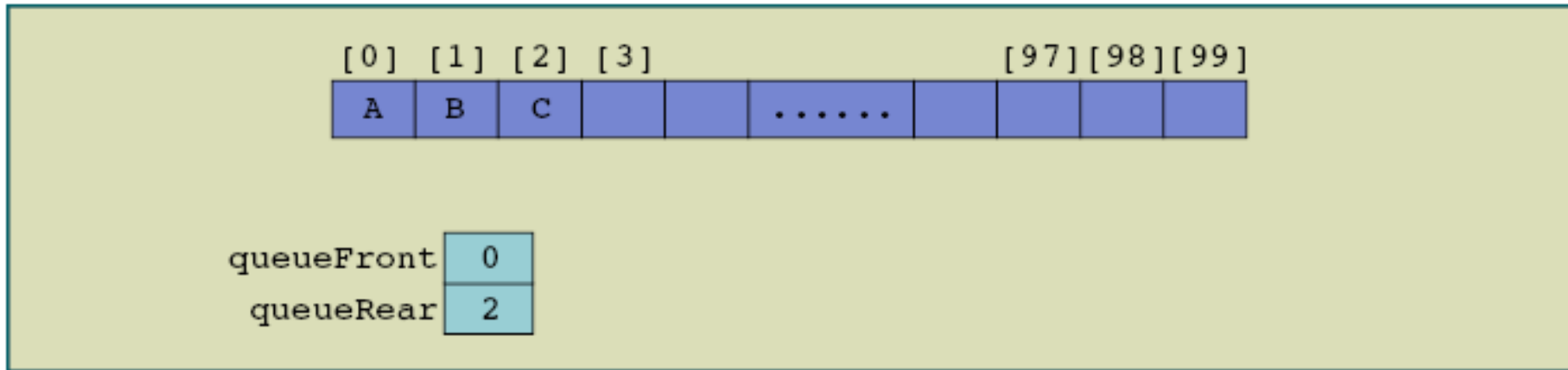
- ♦ **Instance** variables
 - ♦ **An array** to store the queue elements
 - ♦ **queueFront**: keeps track of the first element
 - ♦ **queueRear**: keeps track of the last element
 - ♦ **maxQueueSize**: specifies the maximum size of the queues

Implementation of **Queues** as **Arrays**



Queue after three **addQueue** operations

Implementation of **Queues** as **Arrays**



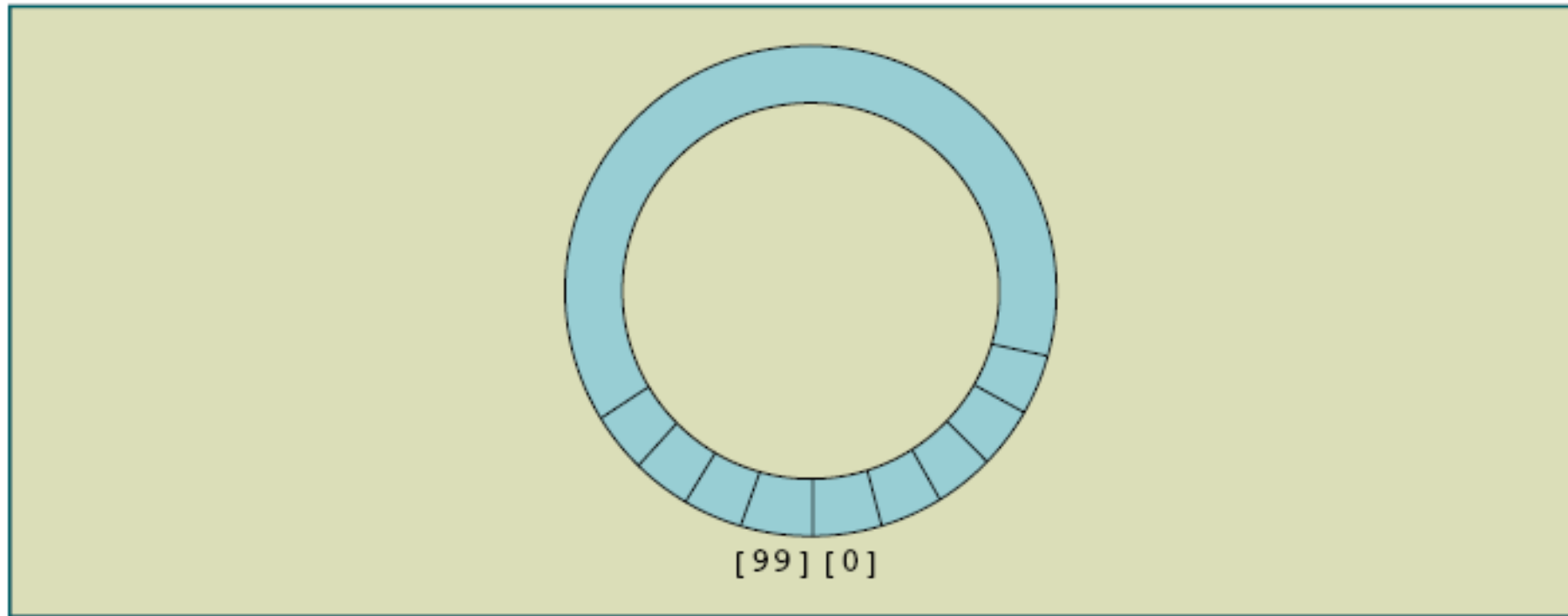
Queue after the **deleteQueue** operation

CORRECTION: **queueFront = 1**

Implementation of **Queues** as **Arrays**

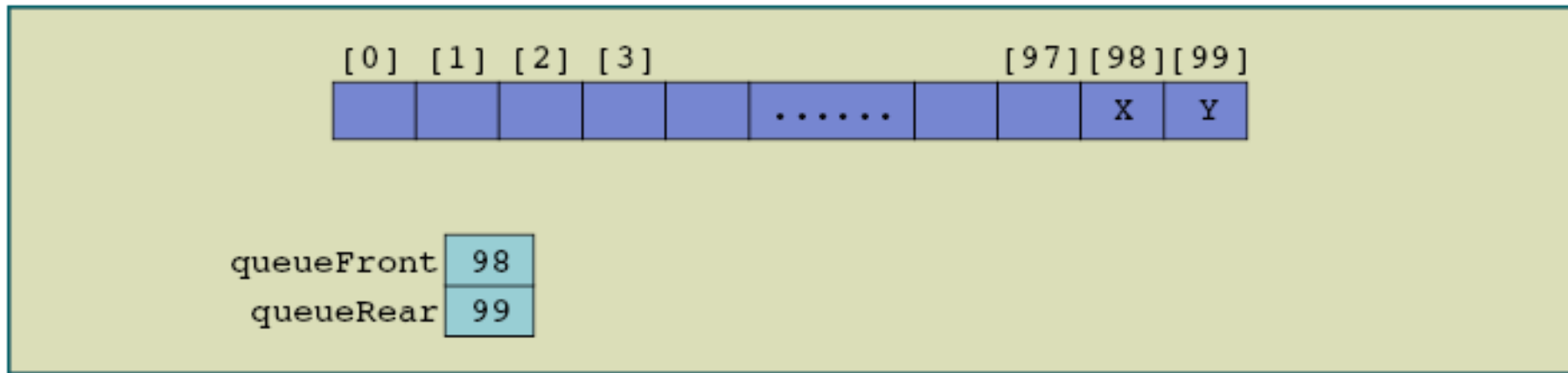
- ♦ **Problems** with this implementation
 - ♦ **Arrays** have fixed sizes
 - ♦ After various insertion and deletion operations, **queueRear** will point to the last array position
 - ♦ Giving the impression that the queue is full
- ♦ **Solutions**
 - ♦ Slide all of the queue elements toward the first array position
 - ♦ Use a circular array

Implementation of **Queues** as **Arrays**



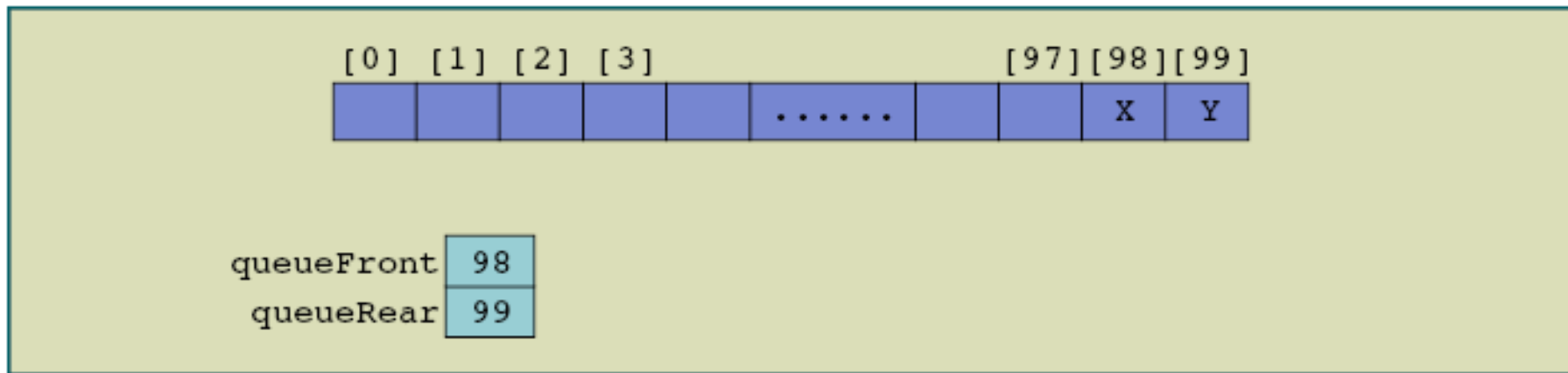
Circular queue

Implementation of **Queues** as **Arrays**



Queue with two elements at positions **98** and **99**

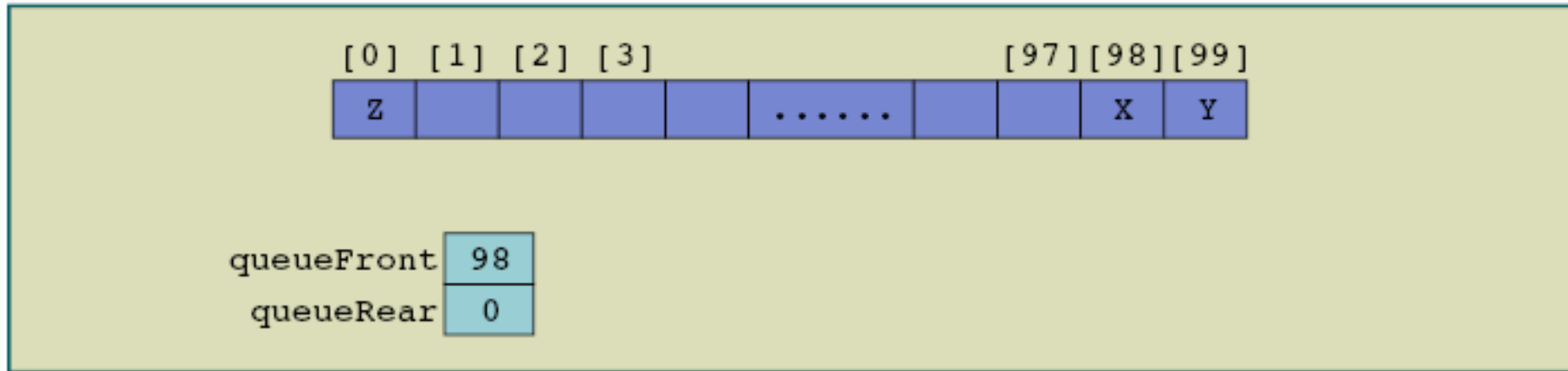
Implementation of **Queues** as **Arrays**



Queue with two elements at positions **98** and **99**

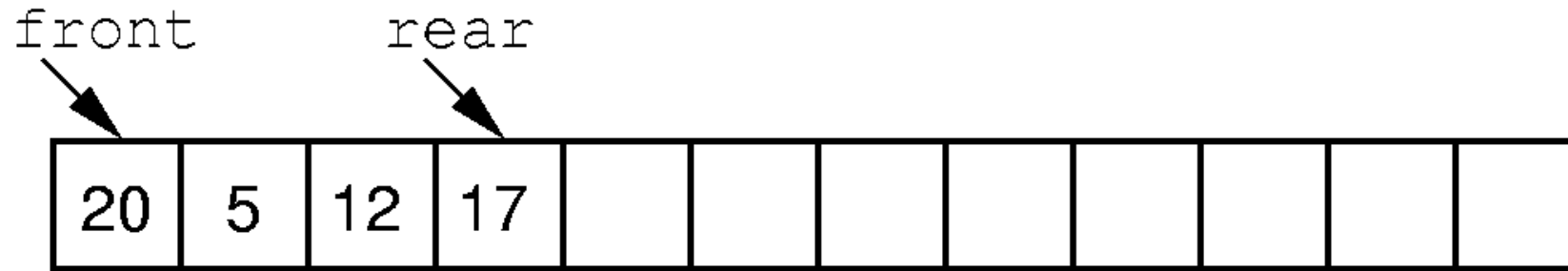
```
i=99, maxSize=100;  
i++; → i=100;  
i=(i+1)%100; → i=(99+1)%100=100%100=0  
i=(i+1)%100=(0+1)%100=1%100=1  
i=(i+1)%100=(1+1)%100=2%100=2
```

Implementation of **Queues** as **Arrays**

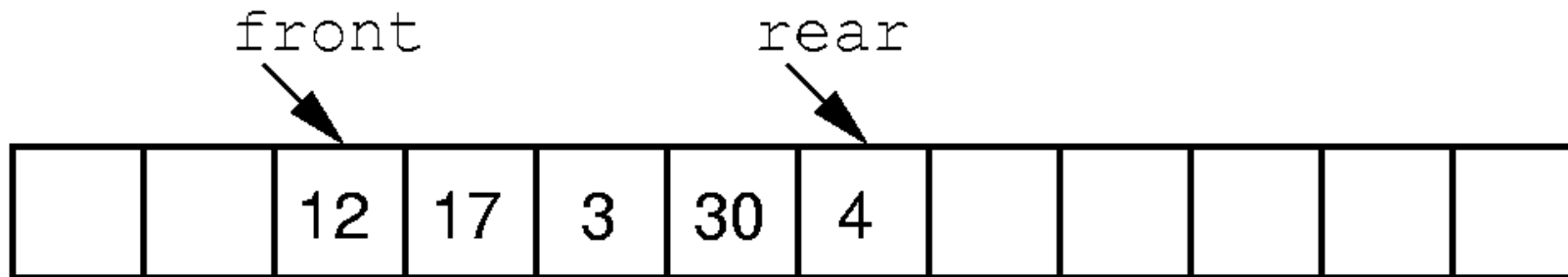


Queue after one more **addQueue** operation

Implementation of **Queues** as **Arrays**

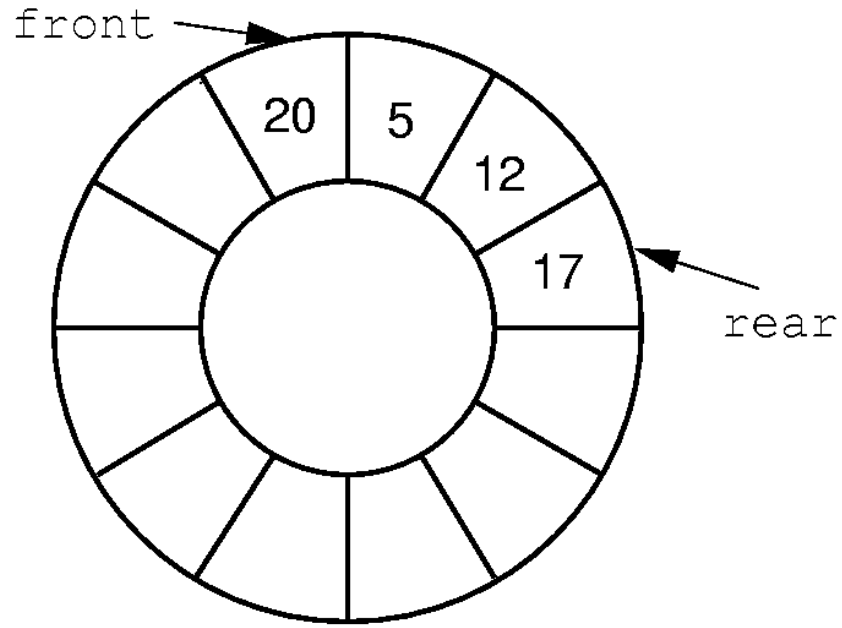


(a)

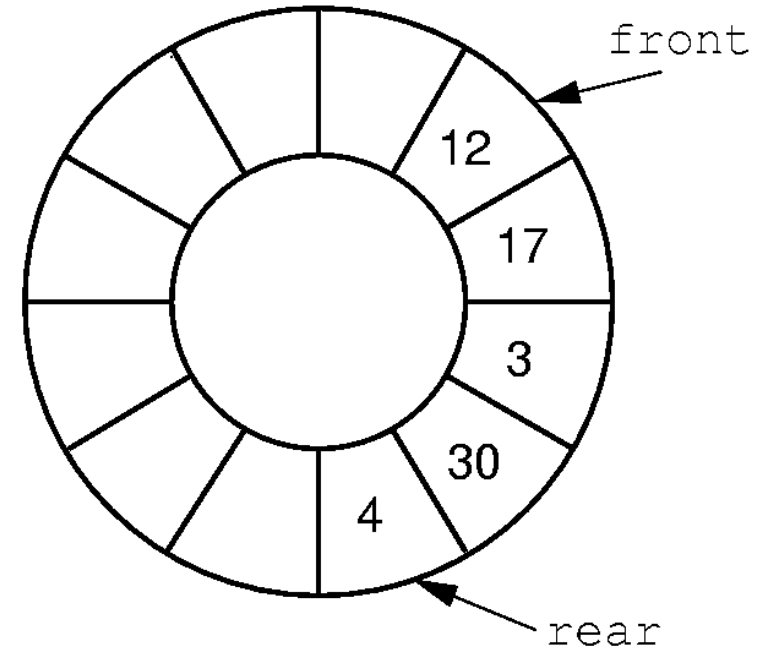


(b)

Implementation of **Queues** as **Arrays**

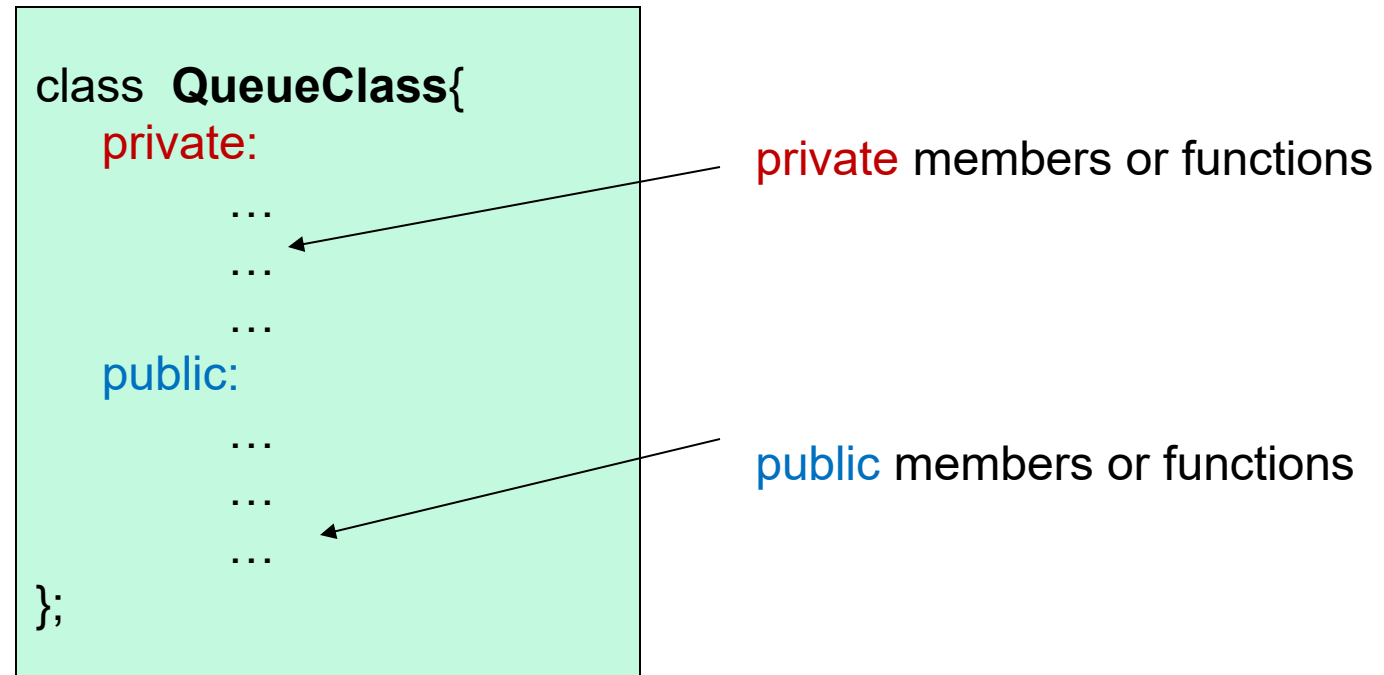


(a)



(b)

QueueClass



QueueClass – data members

```
class QueueClass{  
    private:  
        int maxQueueSize;  
        int *List; //use any data type  
        int queueFront;  
        int queueRear;  
        int count;  
  
    public:  
        ...  
        ...  
        ...  
};
```

private data members

public member functions

- You can use a (alternative of user defined)

QueueClass – functions

- ◆ Default constructor

```
QueueClass() {  
    maxQueueSize = 100;  
    queueFront = 0;  
    queueRear = -1;  
    count = 0;  
    List = new int[maxQueueSize];  
}
```

- ◆ Constructor with parameter

```
QueueClass(int S) {  
    maxQueueSize = S;  
    queueFront = 0;  
    queueRear = -1;  
    count = 0;  
    List = new int[maxQueueSize];  
}
```

QueueClass – functions

♦ function **initilializeQueue**

```
void initializeQueue() {  
    for (int i = queueFront; i < queueRear; i = (i + 1) %  
        maxQueueSize)  
        List[i] = NULL;  
    queueFront = 0;  
    queueRear = -1;  
    count = 0;  
}
```

QueueClass – functions

♦ function **isEmptyQueue**

```
bool isEmptyQueue()  
{  
    return (count == 0);  
}
```

♦ function **isFullQueue**

```
bool isFullQueue()  
{  
    return (count == maxQueueSize);  
}
```

QueueClass – functions

◆ function **front**

```
int front()
{
    if (isEmptyQueue())
    {
        cout<<"The Queue is empty!!!"<<endl;
        return NULL;
    }
    return List[queueFront];
}
```

QueueClass – functions

◆ function **back**

```
int back()
{
    if (isEmptyQueue())
    {
        cout<<"The Queue is empty!!!"<<endl;
        return NULL;
    }
    return List[queueRear];
}
```

Enqueue / addQueue

♦ function **addQueue**

```
void addQueue(int queueElement)
{
    if (isFullQueue())
    {
        cout<<"The Queue is full!!!"<<endl;
        return;
    }
    //circular array → use % op. to advance queueRear
    queueRear = (queueRear + 1) % maxQueueSize;
    count++;
    List[queueRear] = queueElement;
}
```

Deque /deleteQueue

- ◆ Function **deleteQueue**

```
void deleteQueue()  
{  
    if (isEmptyQueue())  
    {  
        cout<<"The Queue is empty!!!"<<endl;  
        return;  
    }  
    count--;  
    List[queueFront] = NULL;  
    //circular array → use % op. to advance queueFront  
    queueFront = (queueFront + 1) % maxQueueSize;  
}
```

Thanks,..

See you next week isA,..